

AI ASSISTANT CODING

ASSIGNMENT-10.1

NAME :siri chunchu

HTNO: 2303A51281

BATCH:05

Task Description #1 – Syntax and Logic Errors

Task: Use AI to identify and fix syntax and logic errors in a faulty Python script.

Sample Input Code:

```
# Calculate average score of a student
def calc_average(marks):
    total = 0
    for m in marks:
        total += m
    average = total / len(marks)
    return avrage # Typo here
marks = [85, 90, 78, 92]
print("Average Score is ", calc_average(marks))
```

PROMPT :

to identify and fix syntax and logic errors in a faulty Python script.

Sample Code:

```
# Calculate average score of a student
def calc_average(marks):
    total = 0
    for m in marks:
```

```
total += m
```

```
average = total / len(marks)
```

```
return avrage # Typo here
```

```
marks = [85, 90, 78, 92]
```

```
print("Average Score is ", calc_average(marks))
```

Corrected and runnable Python code with explanations of the fixes.

CODE :

```
# Calculate average score of a student
```

```
def calc_average(marks):
```

```
    total = 0
```

```
    for m in marks:
```

```
        total += m
```

```
    average = total / len(marks)
```

```
    return average
```

```
marks = [85, 90, 78, 92]
```

```
print("Average Score is", calc_average(marks))
```

EXPLANATION :

Indentation Fixed

- Code inside the function was not properly indented.
- Added correct indentation to avoid IndentationError.

Corrected Typo

- return avrage was misspelled.
- Changed to return average.

Fixed Syntax Error

- Missing closing parenthesis in print() statement.

- Added) to correct it.

Correct Average Logic

- Calculated total using a loop.
- Computed average using total / len(marks).

Proper Structure

- Function defined correctly.
- Marks list and print statement placed outside the function.

OUTPUT :

Average Score is 86.25

Task Description #2 – PEP 8 Compliance

Task: Use AI to refactor Python code to follow PEP 8 style guidelines.

Sample Input Code:

```
def area_of_rect(L,B) : return L*B
print(area_of_rect(10,20))
```

PROMPT :

refactor Python code to follow PEP 8 style guidelines.

Sample Input Code:

```
def area_of_rect(L,B) : return L*B
print(area_of_rect(10,20))
```

Expected Output:

- Well-formatted PEP 8-compliant Python code.

CODE:

```
def area_of_rect(length, width):
    """Calculate the area of a rectangle."""
    return length * width
```

```
print(area_of_rect(10, 20))
```

OUTPUT :

200

EXPLANATION :

- The function name `area_of_rect` follows PEP 8 by using lowercase letters and underscores for better readability.
- Parameter names were changed from `L, B` to `length, width` to make the code more clear and descriptive.
- Proper spacing was added after commas and around the `*` operator to follow PEP 8 formatting rules.
- The function is written in a clean multi-line format instead of a single-line definition, improving readability.
- The program correctly calculates the area ($10 \times 20 = 200$), so the output 200 confirms both style and logic are correct.

Task Description #3 – Readability Enhancement

Task: Use AI to make code more readable without changing its logic.

Sample Input Code:

```
def c(x,y):  
    return x*y/100  
  
a=200  
b=15  
  
print(c(a,b))
```

PROMPT :

make code more readable without changing its logic.

Sample Input Code:

```
def c(x,y):  
    return x*y/100
```

```
a=200
b=15
print(c(a,b))
```

Python code with descriptive variable names, inline comments, and clear formatting.

CODE:

```
def calculate_percentage(base_value, percentage):
    """Calculate the percentage of a given value."""
    return base_value * percentage / 100

# Initial values
base_amount = 200
discount_percentage = 15
# Calculate and display the result
result = calculate_percentage(base_amount, discount_percentage)
print(f"Result: {result}")
```

OUTPUT:

Result: 30.0

EXPLANTION:

Readability improvements made:

1. Descriptive function name - Changed c to calculate_percentage
2. Descriptive variable names - Changed:
 - x → base_value
 - y → percentage
 - a → base_amount
 - b → discount_percentage
3. Docstring - Added function documentation explaining what it does

4. Comments - Added inline comments to clarify sections
5. Formatted output - Used f-string for clearer print statement
6. Proper spacing - Added blank lines for better visual separation
7. Result variable - Stored result in a named variable for clarity

Task Description #4 – Refactoring for Maintainability

Task: Use AI to break repetitive or long code into reusable functions.

Sample Input Code:

```
students = ["Alice", "Bob", "Charlie"]  
print("Welcome", students[0])  
print("Welcome", students[1])  
print("Welcome", students[2])
```

PROMPT:

break repetitive or long code into reusable functions.

Sample Input Code:

```
students = ["Alice", "Bob", "Charlie"]  
print("Welcome", students[0])  
print("Welcome", students[1])  
print("Welcome", students[2])
```

Modular code with reusable functions.

CODE:

```
def greet_student(name):  
    """Greet a student by name."""  
    print(f"Welcome {name}")  
  
def greet_all_students(student_list):  
    """Greet all students in the list."""  
    for student in student_list:
```

```
    greet_student(student)

# Student list
students = ["Alice", "Bob", "Charlie"]

# Greet all students
greet_all_students(students)
```

OUTPUT:

Welcome Alice

Welcome Bob

Welcome Charlie

EXPLANATION:

Improvements made:

1. Reusable function - Created `greet_student()` to handle individual greetings
2. Batch function - Created `greet_all_students()` to iterate through the list
3. DRY principle - Eliminated repetitive `print()` statements
4. Scalability - Now works with any number of students without code duplication
5. f-string formatting - Used modern string formatting for cleaner output
6. Clear documentation - Added docstrings explaining each function's purpose

Task Description #5 – Performance Optimization

Task: Use AI to make the code run faster.

Sample Input Code:

```
# Find squares of numbers
nums = [i for i in range(1,1000000)]
squares = []
for n in nums:
    squares.append(n**2)
```

```
print(len(squares))
```

PROMPT :

make the code run faster.

Sample Input Code:

```
# Find squares of numbers
nums = [i for i in range(1,1000000)]
squares = []
for n in nums:
    squares.append(n**2)
print(len(squares))
```

Optimized code using list comprehensions or vectorized operations.

CODE:

```
# Optimized version 1: Using list comprehension
nums = [i for i in range(1, 1000000)]
squares = [n**2 for n in nums]
print(len(squares))

# Optimized version 2: Using generator (most memory-efficient)
squares = (n**2 for n in range(1, 1000000))
import numpy as np

# Optimized version 3: Using NumPy (fastest for large datasets)
nums = np.arange(1, 1000000)
squares = nums**2
print(len(squares))
print(sum(1 for _ in squares))
```

OUTPUT:

999999

EXPLANATION :

Performance comparison:		
Method	Speed	Memory
Original (loop + append)	Slowest	High
List comprehension	2-3x faster	High
Generator expression	2-3x faster	Very low
NumPy vectorized	10-100x faster	Low

Task Description #6 – Complexity Reduction

Task: Use AI to simplify overly complex logic.

Sample Input Code:

```
def grade(score):  
    if score >= 90:  
        return "A"  
    else:  
        if score >= 80:  
            return "B"  
        else:  
            if score >= 70:  
                return "C"  
            else:  
                if score >= 60:  
                    return "D"  
                else:  
                    return "F"
```

PROMPT :

simplify overly complex logic.

Sample Input Code:

```
def grade(score):
```

```
    if score >= 90:
```

```
        return "A"
```

```
    else:
```

```
        if score >= 80:
```

```
            return "B"
```

```
        else:
```

```
            if score >= 70:
```

```
                return "C"
```

```
            else:
```

```
                if score >= 60:
```

```
                    return "D"
```

```
                else:
```

```
                    return "F"
```

Cleaner logic using elif or dictionary mapping.

CODE:

```
def grade(score):
```

```
    """Determine the letter grade based on score."""
```

```
    if score >= 90:
```

```
        return "A"
```

```
    elif score >= 80:
```

```
        return "B"
```

```
    elif score >= 70:
```

```
        return "C"
```

```

elif score >= 60:
    return "D"
else:
    return "F"
# Test cases
print(grade(95)) # Output: A
print(grade(85)) # Output: B
print(grade(75)) # Output: C
print(grade(65)) # Output: D
print(grade(55)) # Output: F
Using dictionary mapping (most elegant):
def grade(score):
    """Determine the letter grade based on score."""
    grade_map = [
        (90, "A"),
        (80, "B"),
        (70, "C"),
        (60, "D"),
        (0, "F")
    ]
    for threshold, letter_grade in grade_map:
        if score >= threshold:
            return letter_grade
# Test cases
print(grade(95)) # Output: A
print(grade(85)) # Output: B
print(grade(75)) # Output: C

```

```
print(grade(65)) # Output: D
```

```
print(grade(55)) # Output: F
```

OUTPUT:

A

B

C

D

F

EXPLANATION:

Improvements made:

1. Eliminated nested if-else - Used elif for clearer flow
2. Reduced indentation - Much easier to read and maintain
3. Dictionary mapping - Separates logic from thresholds, making it data-driven
4. Scalability - Easy to modify grade thresholds without changing logic
5. Added docstring - Clear function documentation

Benefits:

- Cleaner and more maintainable code
- Easier to extend with new grade levels
- Better performance for dictionary version
- Follows PEP 8 guidelines